Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**ETTIway - Sistem de Navigare Interactivă Campus**

# Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea**
**titlului de** *Inginer*
**în domeniul** *Electronică, Telecomunicații și Tehnologia Informației*
**programul de studii** *Ingineria Sistemelor de Calcul*

**Conducător științific**
**[Profesor Îndrumător]**

**Absolvent**
**Fraticiu Vlad**

**Anul 2025**

## Declaraţie de onestitate academică

Prin prezenta declar că lucrarea cu titlul *ETTIway - Sistem de Navigare Interactivă Campus*, prezentată în cadrul Facultăţii de Electronică, Telecomunicaţii şi Tehnologia Informaţiei a Universităţii "Politehnica" din Bucureşti ca cerinţă parţială pentru obţinerea titlului de *Inginer* în domeniul Inginerie Electronică şi Telecomunicaţii/ Calculatoare şi Tehnologia Informaţiei, programul de studii *Ingineria Sistemelor de Calcul* este scrisă de mine şi nu a mai fost prezentată niciodată la o facultate sau instituţie de învăţământ superior din ţară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referinţe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar şi în traducere proprie din altă limbă, sunt scrise între ghilimele şi fac referinţă la sursă. Reformularea în cuvinte proprii a textelor scrise de către alţi autori face referinţă la sursă. Înţeleg că plagiatul constituie infracţiune şi se sancţionează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor şi măsurătorilor pe care le prezint ca fiind făcute de mine, precum şi metodele prin care au fost obţinute, sunt reale şi provin din respectivele simulări, experimente şi măsurători. Înţeleg că falsificarea datelor şi rezultatelor constituie fraudă şi se sancţionează conform regulamentelor în vigoare.

Bucureşti, Iunie 2025.
Absolvent: Fraticiu Vlad

.....

# Cuprins

**Lista figurilor** . . . iii

**Lista tabelelor** . . . iv
**Lista acronimelor** . . . v

**1. Introducere** . . . 1
1.1. Contextul Proiectului . . . 1
1.2. Motivația și Obiectivele . . . 1
1.3. Structura Documentului . . . 1

**2. Arhitectura Sistemului** . . . 2
2.1. Descriere Generală . . . 2
2.2. Componente Principale . . . 2
2.2.1. Frontend . . . 2
2.2.1.1. Pagina Principală (index.html) . . . 2
2.2.1.2. Stiluri CSS . . . 2
2.2.1.3. Module JavaScript . . . 2
2.2.2. Backend . . . 3
2.2.2.1. Dependențe Principale . . . 3
2.2.2.2. Structura Aplicației . . . 3
2.2.3. Date și Integrations . . . 3
2.2.3.1. Fișiere de Date JSON . . . 3
2.2.3.2. Integrări . . . 3

**3. Tehnologii și Tool-uri Utilizate** . . . 4
3.1. Frontend Stack . . . 4
3.2. Backend Stack . . . 4
3.3. Instrumente de Dezvoltare . . . 4

**4. Funcționalități Implementate** . . . 5
4.1. Funcționalități Completed . . . 5
4.1.1. Hartă Interactivă . . . 5
4.1.2. Interfață Sidebar . . . 5
4.1.3. Sistem Planuri Interioare . . . 5
4.1.4. Repository și Control Versioning . . . 5
4.2. Funcționalități în Dezvoltare . . . 6
4.2.1. Căutare Avansată . . . 6
4.2.2. Localizare Utilizator . . . 6
4.2.3. Sistem de Rutare . . . 6
4.2.4. Sistem de Markere . . . 6

**5. Structura Bazei de Date** . . . 7
5.1. Entități Principale . . . 7
5.1.1. Building (Clădire) . . . 7
5.1.2. Room (Sală) . . . 7
5.1.3. Floor (Etaj) . . . 7

**6. Concluzii și Direcții Viitoare** . . . 8
6.1. Realizări . . . 8
6.2. Provocări și Soluții . . . 8
6.3. Direcții Viitoare . . . 8
6.4. Lecții Învățate . . . 9

**Anexa A. Resurse și Referințe** . . . 10
A.1. Biblioteca JavaScript - Leaflet.js . . . 10
A.2. Framework Backend - Spring Boot . . . 10
A.3. OpenStreetMap . . . 10
A.4. QGIS - Quantum GIS . . . 10

**Anexa B. Instalare și Configurare** . . . 11
B.1. Cerințe de Sistem . . . 11
B.2. Pași de Instalare Backend . . . 11
B.3. Pași de Instalare Frontend . . . 11

**Anexa C. Exemple de Integrare API** . . . 12
C.1. Endpoint-uri Backend Propuse . . . 12
C.1.1. Buildings API . . . 12
C.1.2. Rooms API . . . 12
C.2. Exemplu Request/Response . . . 12
C.2.1. GET /api/buildings - Response Example . . . 12

**Anexa D. Fișiere de Configurare Sursă** . . . 14
D.1. Structura Fișierelor Frontend . . . 14
D.1.1. HTML . . . 14
D.1.2. CSS . . . 14
D.1.3. JavaScript . . . 14
D.1.4. Date . . . 14

**Anexa E. Diagrama Arhitecturii** . . . 15
E.1. Diagrama Componentelor . . . 15
E.2. Flow-ul Datelor . . . 15

# Lista figurilor

# Lista tabelelor

3.1. Tehnologiile Frontend . . . . . 4
3.2. Tehnologiile Backend . . . . . 4

# Lista acronimelor

HA = Harap Alb
PLL = Păsări-Lăți-Lungilă
Sp = Spânul

# Capitolul 1
# Introducere

## 1.1 Contextul Proiectului

ETTIway este o aplicație web interactivă și inovatoare pentru navigarea în campus-ul universitar. Proiectul a fost conceput pentru a facilita deplasarea studenților și vizitatorilor prin facilitățile universitare prin intermediul unei hărți digitale interactive cu funcționalități avansate de căutare și vizualizare.

## 1.2 Motivația și Obiectivele

Navigarea în cadrul unui campus mare și complex poate fi dificilă pentru studenți noi, vizitatori și oaspeți. ETTIway urmărește să:

- Ofere o interfață interactivă și ușor de utilizat pentru explorarea campus-ului
- Furnizeze informații detaliate despre clădiri, săli și alte locații importante
- Permită utilizatorilor să caute rapid clădiri și servicii specifice
- Facilitate navigația prin intermediul unui sistem de rutare
- Integreze date geografice din OpenStreetMap pentru acuratețe și completitudine
- Suporte planuri interioare (floor plans) pentru o înțelegere mai bună a spațiilor

## 1.3 Structura Documentului

Documentul este organizat după cum urmează:

- Capitolul 2 prezintă arhitectura sistemului și componentele principale
- Capitolul 3 detaliază tehnologiile și framework-urile utilizate
- Capitolul 4 descrie funcționalitățile implementate și în curs de dezvoltare
- Capitolul 5 prezintă structura bazei de date
- Capitolul 6 conține concluzii și direcții viitoare de dezvoltare

# Capitolul 2
# Arhitectura Sistemului

## 2.1 Descriere Generală

ETTIway este o aplicație web cu o arhitectură client-server constând din:

- **Frontend**: Aplicație web responsivă construită cu HTML5, CSS3 și JavaScript Vanilla
- **Backend**: Server API construit cu Spring Boot 4.0.5
- **Bază de Date**: PostgreSQL pentru stocarea datelor persistente
- **Hartă Interactivă**: Leaflet.js pentru afișarea și interacțiunea cu harta

## 2.2 Componente Principale

### 2.2.1 Frontend

Frontend-ul constă din mai multe componente:

#### 2.2.1.1 Pagina Principală (index.html)

Pagina principală conține elementele principale ale interfeței:

- Hartă interactivă cu Leaflet.js
- Sidebar-ul lateral cu controale și informații
- Sistemi de navigație și meniu
- Interfață responsive adaptată pentru diverse dimensiuni de ecran

#### 2.2.1.2 Stiluri CSS

- **style.css**: Stiluri principale ale aplicației
- Design responsiv care se adapteaza la desktop, tabletă și telefon
- Palete de culori coerente și interfață ușor de utilizat

#### 2.2.1.3 Module JavaScript

```
- main.js: Punct de intrare al aplicației
- map.js: Gestionarea hărții interactive
- graph-editor.js: Editor pentru date grafice
- indoor.js: Gestiunea planurilor interioare
- routing.js: Calculul și afișarea rutelor
- utils.js: Funcții utilitare
```

### 2.2.2 Backend

Backend-ul este construit cu Spring Boot și oferă:

#### 2.2.2.1 Dependențe Principale

- **Spring Boot Starter Data JPA**: ORM pentru accesul la baze de date
- **Spring Boot Starter Security**: Autentificare și autorizare
- **Spring Boot Starter Web MVC**: Framework web pentru crearea API-urilor
- **PostgreSQL Driver**: Conectivitate la baza de date

#### 2.2.2.2 Structura Aplicației

```
Backend/demo/src/main/java/com/example/demo/
 config/        # Configurații Spring
controller/    # Controllere pentru API endpoints
entity/        # Entități JPA
repository/    # Interfețe Repository
service/       # Logica de business
```

### 2.2.3 Date și Integrations

#### 2.2.3.1 Fișiere de Date JSON

- **campus.sample.json**: Date exemple despre clădiri campus
- **graph_data.json**: Date pentru graful de conectivitate
- **indoor-data.json**: Date despre planuri interioare

#### 2.2.3.2 Integrări

- OpenStreetMap (OSM) pentru date geografice și hărți de bază
- QGIS cu GeoPackage și GeoJSON pentru gestiunea datelor spațiale

# Capitolul 3
# Tehnologii și Tool-uri Utilizate

## 3.1 Frontend Stack

| Tehnologie | Versiune | Utilizare |
|---|---|---|
| HTML5 | - | Structura paginilor |
| CSS3 | - | Stiluri și layout responsiv |
| JavaScript | ES6+ | Logica și interactivitate |
| Leaflet.js | - | Hărți interactive |

Tabela 3.1: Tehnologiile Frontend

## 3.2 Backend Stack

| Tehnologie | Versiune | Utilizare |
|---|---|---|
| Spring Boot | 4.0.5 | Framework web și ORM |
| Java | 17 | Limbaj de programare |
| Spring Security | - | Autentificare/Autorizare |
| Spring Data JPA | - | Acces la baza de date |
| PostgreSQL | - | Bază de date |
| Maven | - | Build tool și dependency manager |

Tabela 3.2: Tehnologiile Backend

## 3.3 Instrumente de Dezvoltare

- **Git**: Versionare și control al codului
- **GitHub**: Repository și colaborare
- **VS Code**: Editor de cod
- **IntelliJ IDEA**: IDE pentru Java/Spring Boot
- **Postman**: Testare API
- **QGIS**: Editare și vizualizare date spațiale

# Capitolul 4
# Funcționalități Implementate

## 4.1 Funcționalități Completed

### 4.1.1 Hartă Interactivă

- Hartă bazată pe Leaflet.js cu suport pentru zoom și pan
- Integrare cu OpenStreetMap pentru date geografice de bază
- Afișarea corectă a pozițiilor GPS și a clădirilor
- Suport pentru diferite layer-e de hartă

### 4.1.2 Interfață Sidebar

- Panoul lateral pentru controale și informații
- Afișare dinamică a detaliilor clădirilor
- Lista clădirilor disponibile
- Controale pentru OSM data
- Cărări responsive care se ascunde pe ecrane mici

### 4.1.3 Sistem Planuri Interioare

- Suport pentru afișarea planurilor interioare (floor plans)
- Fișiere GeoPackage și GeoJSON pentru data management
- Integrare cu QGIS pentru editarea datelor
- Afișarea informatiilor despre săli și holuri

### 4.1.4 Repository și Control Versioning

- Creat repository GitHub cu structura de proiect
- Sistem de versionare și tracking pentru modificări
- Documentație completă în README.md

## 4.2 Funcționalități în Dezvoltare

### 4.2.1 Căutare Avansată

- Câmp de căutare în sidebar
- Filtrare dinamică a clădirilor
- Sugestii în timp real
- Highlight rezultate pe hartă

### 4.2.2 Localizare Utilizator

- Determina poziția utilizatorului prin GPS
- Afișare locației curente pe hartă
- Permisiuni de acces la locație

### 4.2.3 Sistem de Rutare

- Calculul rutei optime între două puncte
- Direcții pas cu pas
- Estimarea timpului de deplasare
- Rute alternative

### 4.2.4 Sistem de Markere

- Markere personalizate pe hartă
- Cluster-are markere pentru o mai bună vizibilitate
- Click handlers pentru detalii

# Capitolul 5
# Structura Bazei de Date

## 5.1 Entități Principale

Baza de date PostgreSQL va conține următoarele entități:

### 5.1.1 Building (Clădire)

```
- id: Identificator unic
- name: Nume clădire
- description: Descriere
- coordinates: Lat/Long
- address: Adresă
- floors: Număr etaje
- services: Servicii disponibile
```

### 5.1.2 Room (Sală)

```
- id: Identificator unic
- building_id: Referință la clădire
- room_number: Numărul sălii
- type: Tip sală (clasă, laborator, etc.)
- capacity: Capacitate persoane
- description: Descriere
```

### 5.1.3 Floor (Etaj)

```
- id: Identificator unic
- building_id: Referință la clădire
- floor_number: Numărul etajului
- floor_plan_url: Link la planul etajului
```

# Capitolul 6
# Concluzii și Direcții Viitoare

## 6.1 Realizări

În cadrul acestui proiect, au fost implementate:

- O aplicație web responsivă și interactivă
- Integrare cu Leaflet.js pentru hărți de înaltă calitate
- Backend robusto cu Spring Boot 4.0.5
- Sistem de securitate și autentificare
- Suport pentru planuri interioare și date spațiale
- Documentație completă a proiectului

## 6.2 Provocări și Soluții

Principalele provocări întâmpinate:

- Integrarea complexă a datelor geografice - rezolvată prin QGIS și GeoJSON
- Performanța hărții cu mulți markeri - optimizată prin clustering
- Cross-browser compatibility - asigurată prin testare pe mai browsere

## 6.3 Direcții Viitoare

Capabilitățile propuse pentru versiuni viitoare:

- Implementare completă a funcționalității de căutare
- Sistem de rutare pe bază de algoritmi de grafuri
- Localizare precisă în timp real
- Suport pentru mai multe limbi
- Integrare social media (share locații)
- Aplicație mobilă native
- Sistem de notificări și alertă
- Analytics și tracking comportament utilizatori

## 6.4 Lecții Învățate

- Importanța planificării arhitecturii înainte de codare
- Valoarea testării progressive și feedback-ului utilizatorilor
- Necesitatea documentației bune pentru mentenabilitate
- Beneficiile utilizării de tool-uri moderne de development
- Importanța versionării codului și colaborării eficiente

# Anexa A
# Resurse și Referințe

## A.1 Biblioteca JavaScript - Leaflet.js

Leaflet.js este o bibliotecă JavaScript lightweight și open-source pentru hărți web interactive. Documentația este disponibilă la `https://leafletjs.com/`.

## A.2 Framework Backend - Spring Boot

Spring Boot este un framework Java modern care simplifică crearea aplicațiilor web și REST APIs. Documentația oficială: `https://spring.io/projects/spring-boot`.

## A.3 OpenStreetMap

OpenStreetMap este o hartă deschisă și colaborativă a lumii. Datele sunt disponibile la `https://www.openstreetmap.org/`.

## A.4 QGIS - Quantum GIS

QGIS este o aplicație GIS open-source pentru procesarea și editarea datelor spațiale. Disponibil la `https://www.qgis.org/`.

# Anexa B
# Instalare și Configurare

## B.1 Cerințe de Sistem

- Browser web modern (Chrome, Firefox, Safari, Edge)
- Java 17 sau mai nou (pentru backend)
- Maven 3.6+ (pentru build)
- PostgreSQL 12+ (pentru baza de date)
- Node.js și npm (opțional, pentru development tools)

## B.2 Pași de Instalare Backend

1. Clonează repository-ul: `git clone https://github.com/Vladeigthteen/ETTIway`
2. Navigează la folderul Backend: `cd Backend/demo`
3. Configurează baza de date PostgreSQL
4. Compilează proiectul: `mvn clean install`
5. Pornește serverul: `mvn spring-boot:run`

## B.3 Pași de Instalare Frontend

1. Navigează la folderul Frontend: `cd Frontend/app`
2. Deschide `index.html` în browser sau pornește un server local
3. Pentru server local cu Python: `python -m http.server 8000`
4. Accesează `http://localhost:8000` în browser

# Anexa C
# Exemple de Integrare API

## C.1 Endpoint-uri Backend Propuse

### C.1.1 Buildings API

```
GET  /api/buildings           - Lista toate clădirile
GET  /api/buildings/{id}      - Obține detalii clădire
POST /api/buildings           - Crează nouă clădire
PUT  /api/buildings/{id}      - Actualizează clădire
DELETE /api/buildings/{id}    - Șterge clădire
```

### C.1.2 Rooms API

```
GET  /api/rooms               - Lista toate sălile
GET  /api/rooms/{id}          - Obține detalii sală
POST /api/rooms               - Crează nouă sală
PUT  /api/rooms/{id}          - Actualizează sală
DELETE /api/rooms/{id}        - Șterge sală
```

## C.2 Exemplu Request/Response

### C.2.1 GET /api/buildings - Response Example


```
[
  {
    "id": 1,
    "name": "Corp B",
    "description": "Clădire academică",
    "latitude": 44.405,
    "longitude": 26.102,
    "address": "Strada Politehnicii, București",
    "floors": 4
  },
  {
    "id": 2,
    "name": "Biblioteca",
    "description": "Biblioteca centrală",
    "latitude": 44.406,
    "longitude": 26.103,
    "address": "Strada Doamnei, București",
    "floors": 3
  }
```

]

# Anexa D
# Fișiere de Configurare Sursă

## D.1 Structura Fișierelor Frontend

Aplicația frontend are următoarele fișiere principale:

### D.1.1 HTML

```
Frontend/app/
 index.html              - Pagina principală
 login.html             - Pagina de login
 admin_test.html        - Pagina admin test
 user_test.html         - Pagina user test
```

### D.1.2 CSS

```
Frontend/app/css/
 style.css              - Stiluri principale
```

### D.1.3 JavaScript

```
Frontend/app/js/
 main.js                - Punct de intrare
 map.js                 - Gestiune hartă
 indoor.js              - Planuri interioare
 routing.js             - Algoritm de rutare
 utils.js               - Funcții utilitare
```

### D.1.4 Date

```
Frontend/app/data/
 campus.sample.json     - Date campus
 indoor-data.json       - Date interioare
 graph_data.json        - Date grafuri

Frontend/app/indoor/CorpB/
 Parter dreapta/        - Planuri etaj
 doors.geojson
  rooms.geojson
 stairs.geojson
   floor.gpkg
```

# Anexa E
# Diagrama Arhitecturii

## E.1 Diagrama Componentelor

Arhitectura sistemului ETTIway este prezentată în figura de mai jos:

## E.2 Flow-ul Datelor

1. Utilizatorul interacționează cu interfața frontend
2. Frontend trimite cereri API la backend
3. Backend procesează cerența și acceseaza baza de date
4. Backend returnează date în format JSON
5. Frontend procesează datele și le afișează pe hartă
6. Hartă folosește Leaflet.js pentru rendering
7. Utilizatorul vede poziții actualizate în timp real